

STM32 HAL - Ściągawka Projektowa

Poniższy dokument stanowi skategoryzowane zestawienie funkcji biblioteki HAL (Hardware Abstraction Layer) dla mikrokontrolerów STM32, wygenerowane na podstawie dostarczonego kodu źródłowego. Zawiera wyjaśnienia parametrów i wzorce użycia.

1. Obsługa Pinów (GPIO) i Opóźnienia

Sterowanie i odczyt stanów (Digital I/O)

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); // lub GPIO_PIN_SET
HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
```

Parametr	Znaczenie
GPIOx (np. GPIOA)	Wskazuje na konkretny port mikrokontrolera (A, B, C, itd.).
GPIO_Pin (np. GPIO_PIN_5)	Numer pinu na danym porcie.
PinState	Stan do ustawienia: GPIO_PIN_SET (stan wysoki, 1) lub GPIO_PIN_RESET (stan niski, 0).

Opóźnienia blokujące

```
HAL_Delay(1000); // Czekaj 1000 ms (1 sekunda)
```

Wskazówka: **HAL_Delay()** używa systemowego timera (SysTick). Jest to funkcja blokująca, co oznacza, że procesor w tym czasie nie wykonuje innej części kodu w głównej pętli.

2. Przerwania Zewnętrzne (EXTI)

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_PIN){
    if(GPIO_PIN == GPIO_PIN_13){
        if(HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)){
            buttonPressed = 0;
        } else {
            buttonPressed = 1;
        }
    }
}
```

Element	Opis / Cel
<code>HAL_GPIO_EXTI_Callback</code>	Funkcja zwrotna (callback) wywoływana automatycznie, gdy nastąpi przerwanie na dowolnym pinie skonfigurowanym jako EXTI.
<code>GPIO_PIN</code>	Parametr przekazujący informację, który pin konkretnie wywołał to przerwanie (np. 13). Zawsze należy sprawdzić (instrukcją IF), czy jest to pin, którego oczekujemy.

3. Komunikacja Szeregowa (UART)

Wysyłanie danych (Transmit)

```
HAL_UART_Transmit(&huart2, (uint8_t *)text_t, 3, 100);
HAL_UART_Transmit(&huart2, (uint8_t*)gear, strlen((char*)gear), HAL_MAX_DELAY);
```

Parametr	Znaczenie
<code>&huart2</code>	Wskaźnik na strukturę konfiguracyjną (uchwyt) portu UART (tutaj UART2).
<code>pData</code> (np. <code>(uint8_t *)text_t</code>)	Wskaźnik na bufor z danymi (rzutowany na <code>uint8_t</code>).
<code>Size</code>	Ilość znaków/bajtów do wysłania (można użyć <code>strlen()</code> przy ciągach znaków).
<code>Timeout</code>	Maksymalny czas oczekiwania na wysłanie (w ms). <code>HAL_MAX_DELAY</code> oznacza czekanie w nieskończoność.

Odbieranie danych z wykorzystaniem przerw (Receive_IT)

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
    HAL_UART_Receive_IT(&huart2, (uint8_t*)text, 1);
    // Przetwarzanie odebranego znaku...
}
```

Wskazówka: `HAL_UART_RxCpltCallback` wywołuje się, gdy zakończy się odbiór zadeklarowanej ilości danych. `HAL_UART_Receive_IT` użyte w środku ponownie "uzbraja" przerwanie na odbiór kolejnego bajtu (wielkość danych: 1).

4. Przetwornik Analogowo-Cyfrowy (ADC)

```
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
raw = HAL_ADC_GetValue(&hadc1);
```

Funkcja / Parametr	Znaczenie
HAL_ADC_Start(&hadc1)	Rozpoczyna konwersję analogowo-cyfrową dla podanego uchwytu (ADC1).
HAL_ADC_PollForConversion(...)	Czeka na zakończenie konwersji (blokuje kod na określony Timeout).
HAL_ADC_GetValue(&hadc1)	Pobiera wyliczoną przez przetwornik, surową wartość (często 12-bitową, od 0 do 4095).

5. Komunikacja z Czujnikami (Magistrala I2C)

Użyta w kodzie do obsługi czujnika temperatury (TMP102) oraz IMU (ICM20648).

```
// Nadawanie do urządzenia (np. wybór rejestru)
buf[0] = REG_TEMP;
ret = HAL_I2C_Master_Transmit(&hi2c1, TMP102_ADDR, buf, 1, 200);

// Odbiór danych z urządzenia
ret = HAL_I2C_Master_Receive(&hi2c1, TMP102_ADDR, buf, 2, 200);
```

Parametr	Znaczenie
&hi2c1	Wskaźnik na uchwyt magistrali I2C.
DevAddress (np. TMP102_ADDR)	Adres czujnika na magistrali. W HAL bibliotekach adres MUSI być przesunięty bitowo w lewo o 1 pozycję (np. 0x48 << 1).
pData (np. buf)	Bufor, z którego wysyłane są dane lub do którego zapisywane są odebrane dane.
Size (np. 1 lub 2)	Ilość przesyłanych bajtów (np. 1 bajt adresu rejestru do zapisu, 2 bajty danych z czujnika do odczytu).

Bitowe przetwarzanie odebranych danych (Bitwise Operations)

Odebrane bajty często muszą zostać połączone w jedną wartość (np. przy sensorze ICM20648):

```
probeX = (buf[0] << 8 | buf[1]) - 2147483648;
```

Wyjaśnienie: Czujniki najczęściej przesyłają starszy bajt (MSB - buf[0]) i młodszy bajt (LSB - buf[1]). Przesunięcie `buf[0] << 8` przesuwa MSB na swoje miejsce, a operacja OR `| buf[1]` "dokleja" LSB, tworząc 16-bitową zmienną.